

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Ramya S S/192421226 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

SET – 5

1. SMART PARKING MANAGEMENT SYSTEM

Problem Understanding

The smart parking system automatically checks parking slot availability, allocates a slot and calculates the parking fee. Efficient intermediate code is required to reduce delays during peak hours.

(i) Intermediate Code for Slot Allocation and Fee Calculation

Method Selected: Three Address Code (TAC), because it represents complex operations using simple instructions.

Small C Code

```
#include <stdio.h>
int main() {
    int slot = 1, hours = 3, fee;
    if (slot > 0) {
        fee = hours * 20;
        printf("Slot Allocated, Fee = %d", fee);
    } else
        printf("No Slot Available");
    return 0;
}
```

Intermediate Code

Step Three Address Code

```
1    t1 = slot > 0
2    if t1 == 0 goto L2
3    t2 = hours * 20
4    fee = t2
5    goto L3
6    L2: print "No Slot"
7    L3
```

Result:

TAC clearly represents slot checking and fee calculation and can be easily converted into machine code.

(ii) Boolean Expressions and Case Statements

Boolean expressions help check conditions such as:

slot_available && payment_verified

Both conditions must be true before allowing a vehicle to park. Case statements can select parking charges based on vehicle type.

Concept	Parking Application
AND	Slot available and payment verified
OR	Normal or reserved slot available
Case	Different rates for car, bike and truck

(iii) Compiler Optimization

Compiler optimization removes unnecessary instructions and reduces execution time.

Important techniques include:

- Constant folding
- Dead code elimination
- Common subexpression elimination
- Control-flow optimization

Result: Faster slot allocation, quicker payment confirmation and lower CPU usage during peak hours.

2. DIGITAL HEALTHCARE APPOINTMENT SYSTEM

Problem Understanding

The healthcare system manages patient verification, doctor availability and appointment booking. Efficient compiler execution is required to reduce confirmation delays.

(i) Intermediate Code for Appointment Scheduling

Method Selected: Three Address Code with procedure calls.

Small C Code

```
#include <stdio.h>
int verify(int id) {
    return id > 0;
}
int main() {
    int patient = 101, available = 1;
    if (verify(patient) && available)
        printf("Appointment Confirmed");
    else
        printf("Appointment Failed");
    return 0;
}
```

Intermediate Code

Step Three Address Code

```
1    param patient
```

Step Three Address Code

```
2    t1 = call verify, 1
3    if t1 == 0 goto L2
4    if available == 0 goto L2
5    print "Appointment Confirmed"
6    goto L3
7    L2: print "Appointment Failed"
```

Result: The intermediate code verifies the patient and checks doctor availability before confirming the appointment.

(ii) Procedure Calls and Modularity

Procedure calls divide the healthcare system into separate modules.

Examples:

verifyPatient()

bookAppointment()

cancelAppointment()

generateReport()

Benefit	Explanation
Modularity	Separate functions perform separate tasks
Reusability	Same function can be used multiple times
Maintenance	Individual modules are easier to modify
Testing	Each procedure can be tested separately

(iii) Compiler Optimization

Optimization reduces execution time and improves system performance.

Techniques such as **constant folding**, **dead code elimination** and **function inlining** can reduce unnecessary operations.

Result: Faster appointment confirmation, patient verification and report generation.

3. AUTOMATED TOLL COLLECTION SYSTEM

Problem Understanding

The toll system identifies vehicles, checks account balance and processes electronic payments. Correct intermediate code and backpatching are required for fast transaction processing.

(i) Intermediate Code for Vehicle Identification and Toll Payment

Method Selected: Three Address Code.

Small C Code

```
#include <stdio.h>
```

```
int main() {
    int rfid = 1, balance = 200, toll = 100;
```

```

    if (rfid == 1 && balance >= toll) {
        balance = balance - toll;
        printf("Toll Paid");
    } else
        printf("Transaction Failed");
    return 0;
}

```

Intermediate Code

Step Three Address Code

```

1    t1 = rfid == 1
2    if t1 == 0 goto L2
3    t2 = balance >= toll
4    if t2 == 0 goto L2
5    balance = balance - toll
6    print "Toll Paid"
7    goto L3
8    L2: print "Transaction Failed"
9    L3:

```

(ii) Significance of Backpatching

Backpatching fills the target address of jump instructions when the target is not known during initial intermediate-code generation.

Example:

```

if rfid == 1 goto _
goto _

```

Later, the blank targets are replaced with the correct labels.

Backpatching Feature Toll Application

True list	Payment processing branch
False list	Transaction rejection
Target filling	Correct control transfer
Benefit	Faster and accurate transaction processing

(iii) Compiler Optimization

Optimization removes unnecessary instructions and reduces execution time.

For example:

```

t1 = balance - toll
balance = t1

```

can be represented directly as:

```

balance = balance - toll

```

Result: Faster toll verification, reduced processing time and improved traffic flow during peak periods.

4. INTELLIGENT FIRE DETECTION AND ALARM SYSTEM

Problem Understanding

The fire detection system monitors smoke and temperature sensors and activates alarms when dangerous conditions are detected. Fast compiler execution is essential because delays can affect safety.

(i) Intermediate Code for Fire Detection

Method Selected: Three Address Code.

Small C Code

```
#include <stdio.h>
int main() {
    int smoke = 1, temp = 80;
    if (smoke == 1 && temp > 70)
        printf("FIRE ALARM");

    else
        printf("Normal");
    return 0;
}
```

Intermediate Code

Step Three Address Code

```
1    t1 = smoke == 1
2    if t1 == 0 goto L2
3    t2 = temp > 70
4    if t2 == 0 goto L2
5    print "FIRE ALARM"
6    goto L3
7    L2: print "Normal"
8    L3:
```

(ii) Case Statements for Emergency Conditions

Case statements efficiently select an action based on the fire level.

switch(fire_level)

Fire Level Action

1	Normal monitoring
2	Warning alarm
3	Emergency alarm
Default	Sensor error

This avoids writing many complex nested conditions.

(iii) Compiler Optimization

Optimization reduces the number of instructions executed by the fire detection system.

Techniques include:

- Boolean optimization
- Dead code elimination
- Constant folding
- Control-flow optimization

Result: Faster sensor processing and immediate alarm activation, improving industrial safety.

5. AI-BASED CUSTOMER SUPPORT CHATBOT

Problem Understanding

The chatbot receives customer queries, identifies the request and generates an appropriate response. Efficient compiler execution is required to provide fast responses for large numbers of users.

(i) Intermediate Code for Query Processing

Method Selected: Three Address Code with procedure calls.

Small C Code

```
#include <stdio.h>

void response(int intent) {
    if (intent == 1)
        printf("Order Information");
    else
        printf("Contact Support");
}

int main() {
    int user = 101, intent = 1;
    if (user > 0)
        response(intent);
    return 0;
}
```

Intermediate Code

Step Three Address Code

```
1    t1 = user > 0
2    if t1 == 0 goto L2
3    param intent
4    call response, 1
5    goto L3
6    L2: print "Invalid User"
7    L3:
```

(ii) Procedure Calls and Modularity

Procedure calls divide chatbot operations into separate modules.

Examples:

verifyUser()

analyzeQuery()

detectIntent()

generateResponse()

Benefit	Chatbot Application
Modularity	Separate query-processing tasks
Reusability	Functions reused for different queries
Maintenance	Easy to update individual modules
Scalability	New chatbot features can be added

(iii) Compiler Optimization

Compiler optimization reduces unnecessary computations and improves response speed.

Important techniques include:

- Constant folding
- Dead code elimination
- Function inlining
- Common subexpression elimination

Result: Faster response generation, lower resource usage and better performance when many customers use the chatbot simultaneously.

COMPARATIVE SUMMARY

System	Main Compiler Concept	Main Benefit
Smart Parking	TAC, Boolean, Case	Faster slot allocation
Healthcare	TAC, Procedure Calls	Faster appointment confirmation
Toll Collection	TAC, Backpatching	Faster payment processing
Fire Detection	TAC, Boolean, Case	Faster emergency response
AI Chatbot	TAC, Procedure Calls	Faster response generation

OVERALL CONCLUSION

Compiler design concepts are highly useful in modern industry applications. Intermediate code provides an efficient representation of programs, while Boolean expressions, case statements, procedure calls and backpatching help generate correct control flow.

Compiler optimization reduces execution time and resource usage. Therefore, an optimized compiler can improve the speed, reliability, scalability and real-time performance of smart parking, healthcare, toll collection, fire detection and AI chatbot systems.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

